

APPLIED MATH FOR COMPUTER SCIENCE

CPCS 212

Lab 6: **Roots of Equations:** **Newton's Method and Secant Method**

Newton's Method idea

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)} \quad (1)$$

and can be expressed in the following form:

$$x_{i+1} - x_i = -\frac{f(x_i)}{f'(x_i)} \quad (2)$$

2. Newton's Method

Algorithm

Input: Continuous function $f(x)$ and first derivative

Initial guess x_0

Output: root of function $f(x)$ near x_0

Assumptions:

$f(x)$ is continuous

$f'(x_0) \neq 0$

Steps:

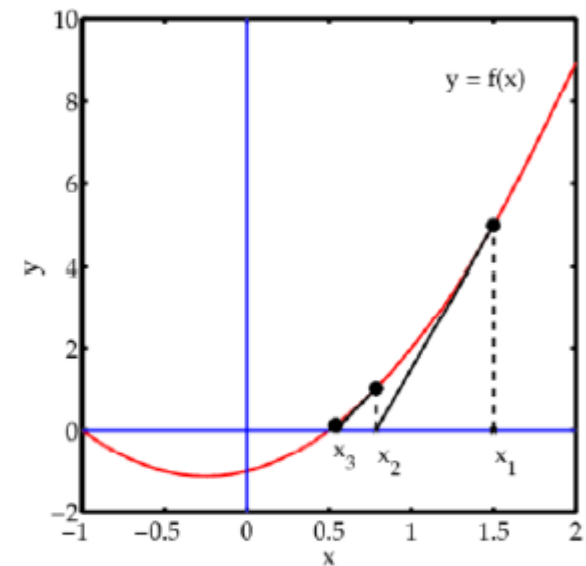
$x_i = x_0$

Loop

1. Evaluate $f(x_i)$ if $f(x_i) = 0$, stop x_i is the root

2. Compute next point $x_i \leftarrow x_i - \frac{f(x_i)}{f'(x_i)}$

End loop



Rules for differentiation

1. $\frac{d}{dx}(c) = 0$, where c is a constant

2. $\frac{d}{dx}(x^n) = nx^{n-1}$, where n is any real number

3. $\frac{d}{dx}(e^x) = e^x$

4. $\frac{d}{dx}(\ln x) = \frac{1}{x}$, for $x > 0$

5. $\frac{d}{dx}(\sin x) = \cos x$

6. $\frac{d}{dx}(\cos x) = -\sin x$

Open the diary file
Lab6_yourname.txt

Exercise 1

1. Use the four functions m-files $f1.m$, $f2.m$, $f3.m$ and $f4.m$ from Lab 5, Exercise 5 in this lab.
2. Create another four functions $df1.m$, $df2.m$, $df3.m$ and $df4.m$ to find the derivative of each function

1-(a)

Name	Formula	Derivative formula
f1	x^2-9	$yprime = 2 * x$
f2	$x^5- x-1$	$yprime = 5 * x^4 - 1$
f3	$3 * x - \exp(-x)$	$yprime = 3 + \exp(-x)$
f4	$2 * \cos(3 * x) - \exp(x)$	$yprime = - 6 * \sin(3 * x) - \exp(x)$

f1.m

```
function [y]=f1(x)
% [y]=f1(x) computes  $y=x^2-9$ 
% your name and the date
y=x.^2-9;
end
```


df1.m

```
function [yprime]=df1(x)
% [yprime]=df1(x) computes the derivative of  $y=x^2-9$ 
% your name and the date
yprime=2*x;
end
```

1-(b)

Use the `help f1` command to confirm that your comments include at least the formula for `f1`, and similarly for `f2`, `f3`, `f4` and their derivatives

```
>> help f1    >> help df1
>> help f2    >> help df2
>> help f3    >> help df3
>> help f4    >> help df4
```

1-(c)

- What are the values of the function and derivative at $x=2$ for each of the four functions?

```
>> [fx] = f1(2)
>> [dfx] = df1(2)
```

OR

```
>> [fx] = feval('f1',2)
>> [dfx] = feval('df1',2)
```

```
fx =   ???
dfx =   ???
```

Exercise 2

In this exercise, you will get down to the task of writing Newton's method as a function m-file. In this m-file, you will see how to use a *variable* number of arguments in a function to simplify later calls. The *size of the error* and the *maximum number of iterations* will be optional arguments. When these arguments are omitted, they take on their default values.

2-(a)

- Open a new, empty m-file named **newton3.m**.

2-(b)

- Start off the function with its signature line and follow that with comments.
- The comments should:
 - repeat the signature as a comment,
 - contain a line or two explaining what the function does, explain what each of the parameters mean, and include your name and the date.

```
function [x,numIts]=newton3(f,df,x0,e,maxIts)
%[x,numIts]=newton3(f,df,x0,e,maxIts)
% f is the name of a function
% df is the derivative of f
% x0 is the initial guess
% numIts is a number of iterations
% e is a pre-specified error tolerance
(optional)
% maxIts is a maximum number of
iterations(optional)
% x is the final solution
% your name and the date
```

2-(c)

- The special Matlab variable *nargin* gives a count of the number of parameters that were used in the function call.
- Allows the parameter *e* and *maxIts* to be omitted when its default values is satisfactory.

```
if nargin < 5
    maxIts=100;
end

if nargin < 4
    maxIts=100;
    e=1.0e-6 ;
end
```


2-(d)

- Put the initial value of x and then start off the loop :

```
x=x0 ;  
for numIts = 1 : maxIts
```

2-(e)

- Evaluate the function and its derivative at the current point x using *feval*, as was done in `bisect.m`.

```
fx    = feval(f,x) ;  
dfx  = feval(df,x) ;
```

2-(f)

- Calculate the new x by $x_1 = x - (f(x)/f'(x))$

$$x_1 = x - (fx/dfx) ;$$

2-(g)

- Finish the loop and the m-file with the following statements:

```
errorEstimate = abs((x-x1)/x1);  
    x=x1;  
    fprintf('Iteraiton: %f    x: %f    errorEstimate: %e  
\n',numIts,x,errorEstimate);  
  
    if (errorEstimate<e) || (abs(fx)<e)  
        fprintf('The solution is: %f ',x); return;  
    end  
  
end %end of Loop  
  
warning('newton: maximum number of iterations exceeded.');
```

end

```
function [x,numIts]=newton3(f,df,x0,e,maxIts)
%[x,numIts]=newton3(f,df,x0,e,maxIts)
% f is the name of a function
% df is the derivative of f
% x0 is the initial guess
% numIts is a number of iterations
% e is a pre-specified error tolerance (optional)
% maxIts is a maximum number of iterations(optional)
% x is the final solution
% your name and the date

%set the default number of iterations
if nargin < 5
    maxIts=100;
end

%set the default number of iterations and error tolerance
if nargin < 4
    maxIts=100;
    e=1.0e-6 ;
end
```

```

x = x0;
fprintf('Iterations details:');
for numIts = 1 : maxIts
    %Solve equation and its derivative using x
    fx = feval(f,x) ;
    dfx = feval(df,x);
    %find the new x by  $x = x - (f(x)/f'(x))$ 
    x1 = x - (fx/dfx);
    %find error estimation using new x
    errorEstimate = abs((x-x1)/x1);
    x=x1;
    %print iteration information
    fprintf('Iteraiton: %f    x: %f    errorEstimate: %e
\n',numIts,x,errorEstimate);

    if (errorEstimate<e) || (abs(fx)<e)
        fprintf('The solution is: %f ',x);
        return;
    end
end %end of Loop

warning('newton: maximum number of iterations exceeded. ');
x= []; numIts = 'No solution is founded';
end

```

2-(h)

- Test your *newton3* function on the function *f1* that you wrote in the previous exercise. Start from an initial guess of $x=0.1$.

```
>> [approxRoot numIts]=newton3('f1','df1',0.1);
```

```
>> [approxRoot numIts]=newton3('f1','df1',0.1,eps)
```

```
>> [approxRoot numIts]=newton3('f1','df1',0.1,eps,10)
```

- The correct solution, of course, is $x=3$.

2-(i)

- Try again, starting at $x = -0.1$, you should get $x = -3$

```
>> [approxRoot numIts]=newton3('f1','df1',-0.1);
```


2-(j) Fill in the following table.

Name	Formula	Initial guess	approxRoot (r)	No. Steps
f1	x^2-9	0.1		
f2	x^5-x-1	10		
f3	$3*x-\exp(-x)$	10		
f4	$2*\cos(3*x)-\exp(x)$	0.1		
f4	$2*\cos(3*x)-\exp(x)$	1		

Secant's Method idea:

Calculate the next estimate of the root from two initial guesses

$$x_{i+1} = x_i - \frac{f(x_i)(x_i - x_{i-1})}{f(x_i) - f(x_{i-1})}$$

Find the absolute relative approximate error

$$|\epsilon_a| = \left| \frac{x_{i+1} - x_i}{x_{i+1}} \right|$$

3. Secant Method

Algorithm Input:

Continuous function $f(x)$

Two initial guess x_1, x_2

Output:

root of function $f(x)$ near x_1 and x_2

Assumptions:

$f(x)$ is continuous

$$f(x_1) \neq f(x_2)$$

Steps:

Evaluate $f(x_1)$ if $f(x_1) == 0$, stop x_1 is the root

Evaluate $f(x_2)$ if $f(x_2) == 0$, stop x_2 is the root

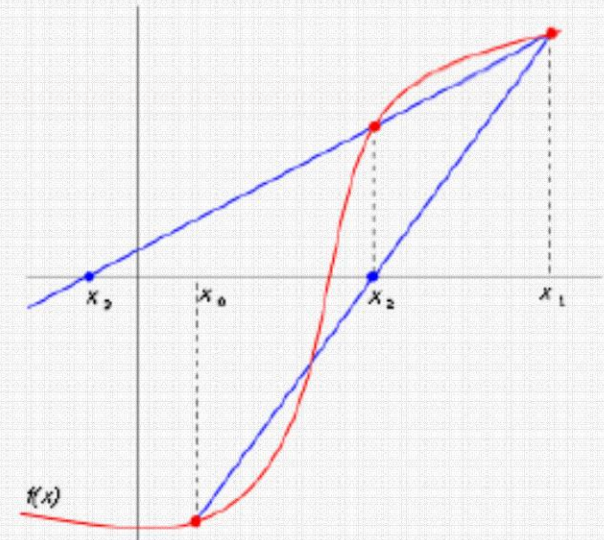
Loop

$$1. x \leftarrow x_2 - f(x_2) \frac{x_2 - x_1}{f(x_2) - f(x_1)}$$

2. Evaluate $f(x)$ if $f(x) == 0$, stop x is the root

$$3. \begin{aligned} x_1 &\leftarrow x_2 \\ x_2 &\leftarrow x \end{aligned}$$

End loop



```
function [x,numIts]=secant(f,x1,x2,tol,maxIts)
%[x,numIts]=secant(f,x1,x2,tol,maxIts)
% Secant Method
% x is the final solution
% numIts: number of iteration
% f: function equation in variable x
% [x1,x2]: searched section (the initial guesses)
% tol is a pre-specified error tolerance (optional)
% maxIts is a maximum number of iterations(optional)

%set the default number of iterations
if nargin < 5
    maxIts = 100;
end
%set the default number of iterations and error
tolerance
if nargin < 4
    maxIts = 100;
    tol = 1.0e-6;
end
```

```
fprintf('secant method\n');  
fprintf('k\t\tx\t\tt\tx\t\tterror\n');  
  
%Solve equation using x1 and x2  
fx1 = feval(f,x1);  
fx2 = feval(f,x2);  
  
%check solution by error tolerance  
if abs(fx1)<tol  
    x = x1;  
    fprintf('The root is: %f',x);  
    return;%solution is founded  
elseif abs(fx2)<tol  
    x = x2;  
    fprintf('The root is: %f',x);  
    return;%solution is founded  
end  
  
if abs(fx1) < abs (fx2)  
    temp = x1; x1 = x2; x2 = temp;  
    temp = fx1; fx1 = fx2; fx2 = temp;  
end
```

```

numIts=0;
while numIts < maxIts
    numIts = numIts + 1;

    %calculate the new x by secant method
    x=x1-fx1*(x1-x2)/(fx1-fx2);
    %solve the equation using thr new x
    fx=feval(f,x);
    %print iteration information
    fprintf('\t%d \t%7.4f\t\t%7.4f\t\t%8.4f %%
\n',numIts,x,fx, abs(fx)*100);

    if (abs(fx)<tol) || abs((x-x1)/x)< tol
        fprintf('The solution is: %f ',x);
        return;%the solution is founded
    else
        x1 = x2; x2 = x; %update the values
        fx1 = feval(f,x1);
        fx2 = feval(f,x2);
    end
end
warning('secant: maximum number of iterations exceeded. ');
x= []; numIts = 'No zeros in given interval';
end

```

Call the program:

```
>> [x,n] = secant('f1',5,10)
```

```
>> [x,n] = secant('f1',5,10,eps)
```

```
>> [x,n] = secant('f1',5,10,eps,5)
```

Homework

- Find the root of the equation $f(x) = x^2 - 3$ with accuracy of 0.01 in the interval $[1, 2]$ using bisection method.
- Find the root of the equation $F(x) = x^2 - 4x - 7$ with initial guess 4 with accuracy up to 4 decimal places. ($\epsilon \leq 0.005$)
- Find the root of the equation $x^3 = 20$ using secant method in the interval $[2, 3]$.